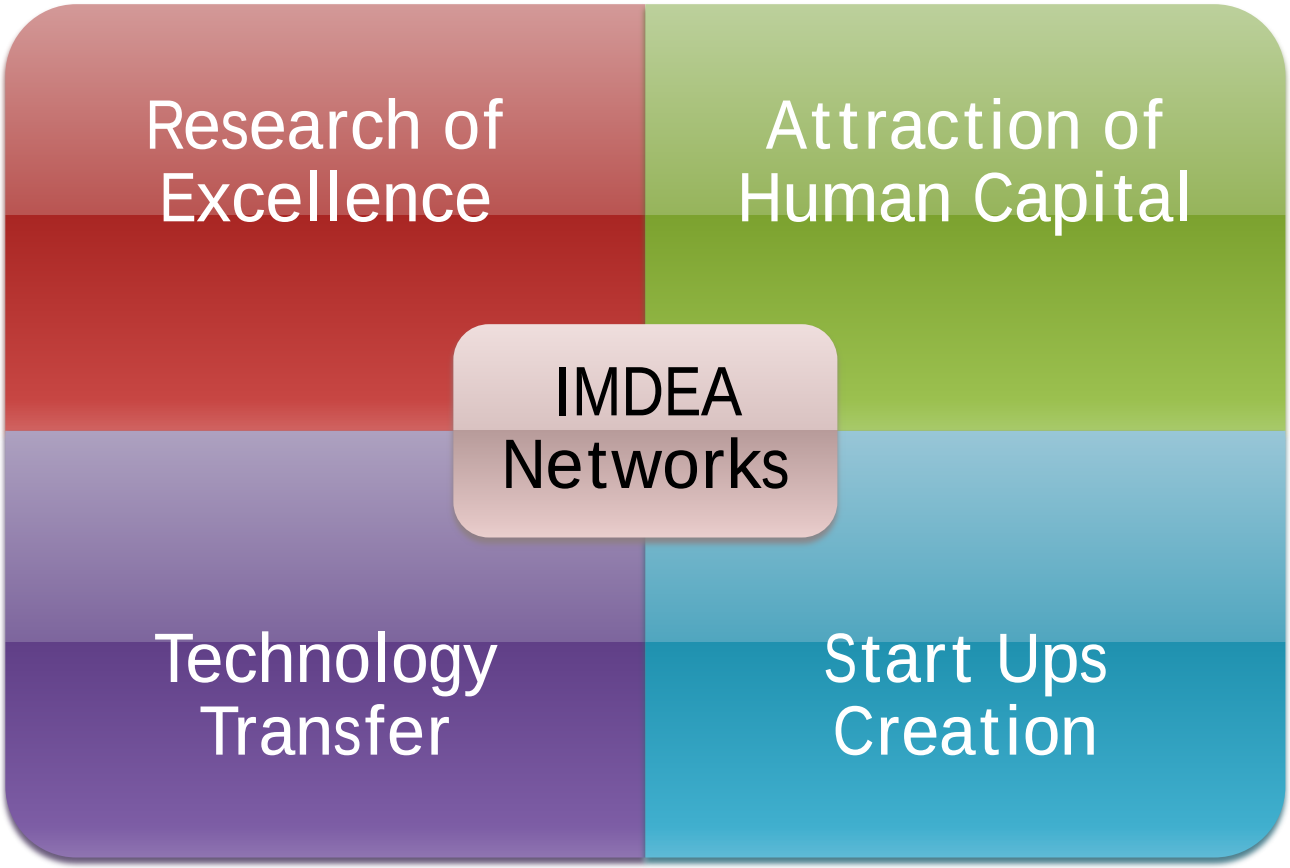
Atomic Appends: Selling Cars and Coordinating Armies with Multiple Distributed Ledgers

Antonio Fernandez Anta (IMDEA Networks Institute)
Chryssis Georgiou (U. Cyprus)
Nicolas Nicolaou (Algolysis LTD)

Developing the Science of Networks

Introduction to IMDEA Networks

- Public research center, founded in 2006, operational since 2009



Highlights

- Steady production of more than 100 papers/yr
- Constant presence in top conferences:
  - In INFOCOM 2017 : 10 papers accepted, 7 TPC (3 Distinguished) Members, 3 best presentation in session
  - In INFOCOM 2018: 6 papers accepted, 5 TPC Distinguished Members
  - General co-Chair of SIGCOMM 2018 Sergey Gorinsky
- Spanish institutions with highest number of papers in IEEE/ACM Trans. on Networking and IEEE INFOCOM Conference

CSRankings

CSRankings: Computer Science Rankings

CSRankings is a metrics-based ranking of top computer science institutions around the world. **Click on a triangle** (▶) to expand areas or institutions. **Click on a name** to go to a faculty member's home page. **Click on a pie** (the ◔ after a name or institution) to see their publication profile as a pie chart. **Click on a Google Scholar icon** (🎓) to see publications, and **click on the DBLP logo** (▶) to go to a DBLP entry.

Rank institutions in Europe by publications from 2009 to 2019

All Areas [off | on]

AI [off | on]

- ▶ Artificial intelligence ☐
- ▶ Computer vision ☐
- ▶ Machine learning & data mining ☐
- ▶ Natural language processing ☐
- ▶ The Web & information retrieval ☐

Systems [off | on]

- ▶ Computer architecture ☐
- ▶ Computer networks ☑
- ▶ Computer security ☐
- ▶ Databases ☐
- ▶ Design automation ☐
- ▶ Embedded & real-time systems ☐
- ▶ High-performance computing ☐
- ▶ Mobile computing ☑
- ▶ Measurement & perf. analysis ☑
- ▶ Operating systems ☐

#	Institution	Count	Faculty
1	▶ ETH Zurich ◔	5.0	13
2	▶ Max Planck Institute ◔	4.1	7
3	▶ University of Cambridge ◔	3.5	13
4	▶ EPFL ◔	2.8	9
5	▶ IMDEA Networks Institute ◔	2.5	8
6	▶ University College London ◔	2.4	6
6	▶ University of Edinburgh ◔	2.4	5
8	▶ Technion ◔	2.2	12
9	▶ Hebrew University of Jerusalem ◔	2.0	4
10	▶ TU Berlin ◔	1.9	3
11	▶ TU Delft ◔	1.8	7
11	▶ University of Oxford ◔	1.8	4
13	▶ University of Crete ◔	1.7	2
14	▶ Imperial College London ◔	1.5	4

Career Opportunities

- We are hiring!!
  - All faculty ranks
  - Postdocs
  - PhD students
  - Interns

- Check https://www.networks.imdea.org/careers

Atomic Appends: Selling Cars and Coordinating Armies with Multiple Distributed Ledgers

Antonio Fernandez Anta (IMDEA Networks Institute)
Chryssis Georgiou (U. Cyprus)
Nicolas Nicolaou (KIOS, U. Cyprus & Algolysis)

Developing the Science of Networks

What are blockchains?

- Block chains and cryptocurrencies may have the power to change society
- But when experts are asked, they often get very technical about crypto, mining, smart contracts

ETHEREUM: A SECURE DECENTRALISED GENERALISED TRANSACTION LEDGER

EIP-150 REVISION (759dccd - 2017-08-07)

DR. GAVIN WOOD
FOUNDER, ETHEREUM & ETHCORE
GAVIN@ETHCORE.IO

ABSTRACT. The blockchain paradigm when coupled with cryptographically-secured transactions has demonstrated its utility through a number of projects, not least Bitcoin. Each such project can be seen as a simple application on a decentralised, but singleton, compute resource. We can call this paradigm a transactional singleton machine with shared-state.

Ethereum implements this paradigm in a generalised manner. Furthermore it provides a plurality of such resources, each with a distinct state and operating code but able to interact through a message-passing framework with others. We discuss its design, implementation issues, the opportunities it provides and the future hurdles we envisage.

Blockchain Formalization



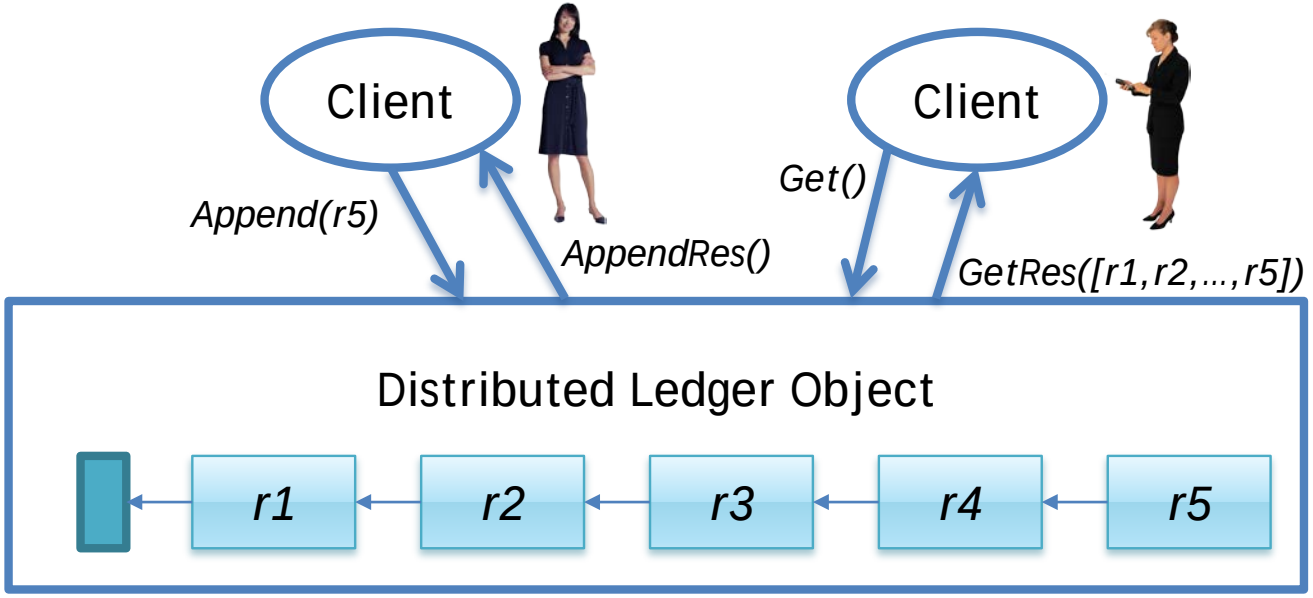
There is need for formalization

[Herlihy's Keynote at PODC 2017]

and possibly use the decades of experience in DC

Distributed Ledger Objects

- We contributed to this formalization defining the Distributed Ledger Object (DLO) [FGKN 2018]
  - Concurrent object that stores a totally ordered sequence of *records:* ***[r1, r2, r3, …, rn]***
  - Clients can ***Append(r)*** a new record ***r*** or ***Get()*** the whole sequence of records (*operations*)
  - Start and end of an operation are matching events



DLO Centralized Implementation

Centralized DLO

Init: *S := []*

Upon invocation *Append(r)* do

if *(r not in S)*

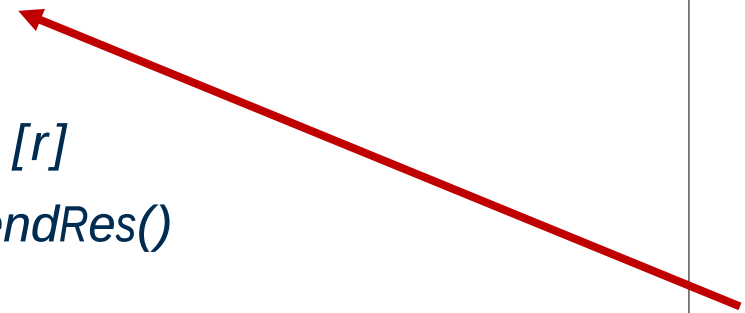
then

S := S + [r]

respond *AppendRes()*

Upon invocation *Get()* do

respond *GetRes(S)*



Appends of the same record *r* are idempotent

institute IMDEA networks

Validated DLO Centralized Implementation

Validated Centralized DLO (*Valid*())

Init: *S := []*

Upon invocation *Append(r)* do

if *((r not in S) and Valid(S+[r]))*

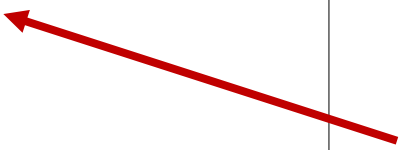
then

S := S + [r]

respond *AppendRes()*

Upon invocation *Get()* do

respond *GetRes(S)*



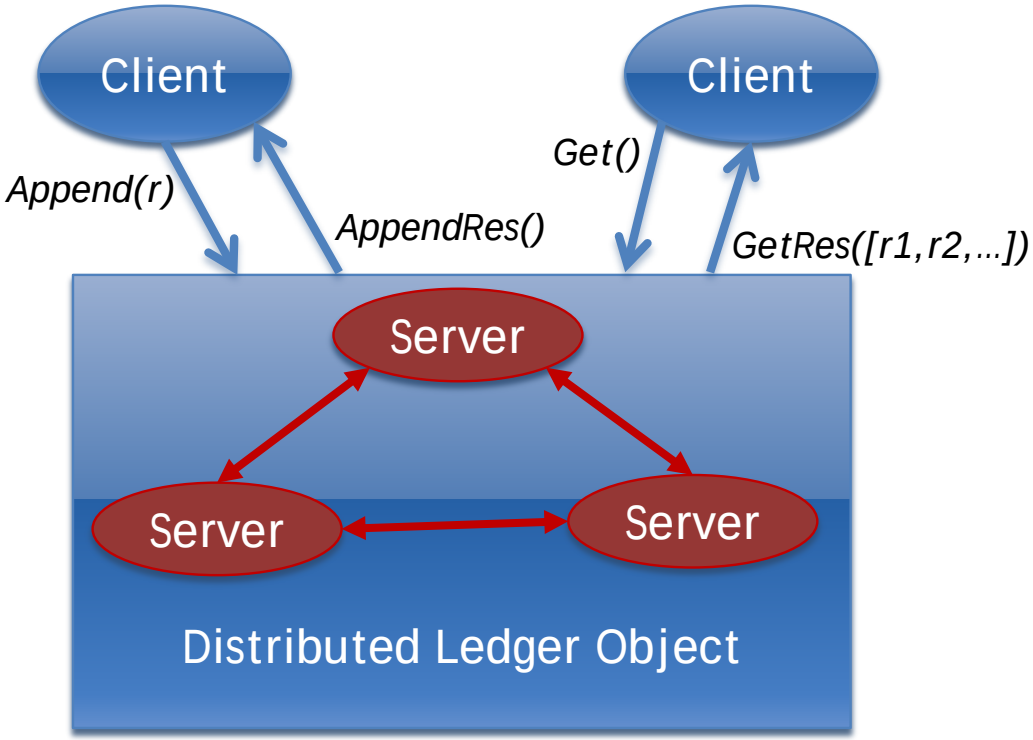
Validation can check hash links, transaction correctness, and run Smart Contracts

- Validity check before appending a record

institute IMDEA networks

Distributed Implementations

- We are interested in distributed implementations of DLO
- A set of servers collaborate to provide the service and properties of a ledger



institute IMDEA networks

Linearizable DLO

- There is a permutation σ of the operations of every execution of a DLO so that:
  - All records appended are seen in the same order by every client: Strong prefix [Anceaume et al., 2018]
  - Atomic consistency (linearizability): The order σ respects the execution order of the operations ("looks like" a centralized ledger)
- In [FGKN 2018] and [CFGN 2019] we showed how to implement a linearizable DLO using a total order broadcast service when clients and servers can fail (crash or Byzantine)

institute IMDEA networks

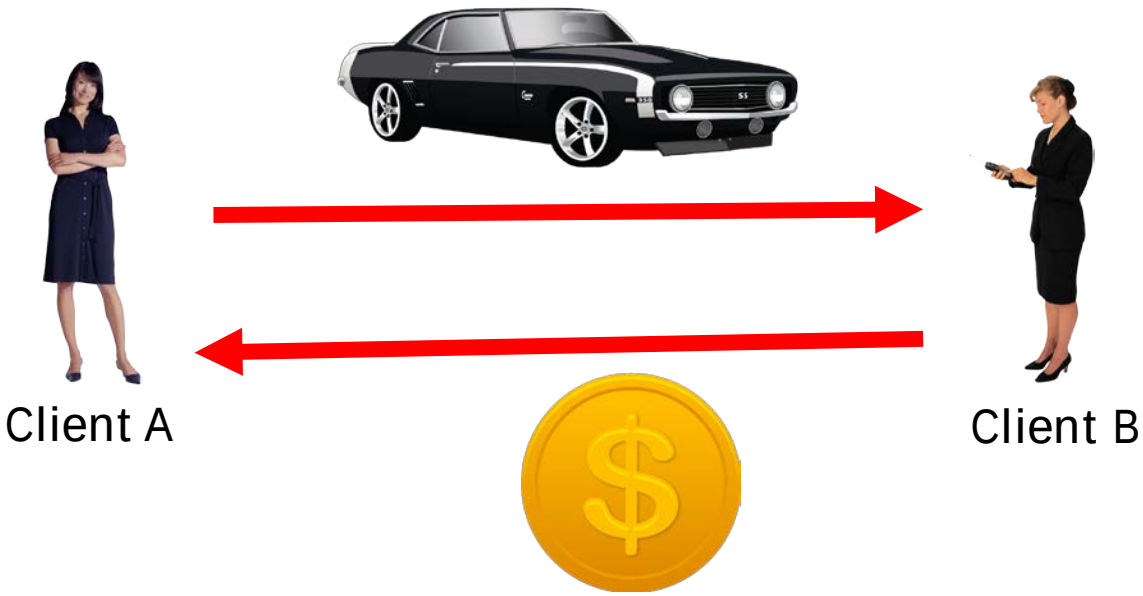
Interconnection of Blockchains

- A convergence to one single blockchain is very unlikely
- Interconnection of blockchains is unavoidable

- One of the simplest operation involving multiple blockchains is Atomic Swap

Atomic Swap

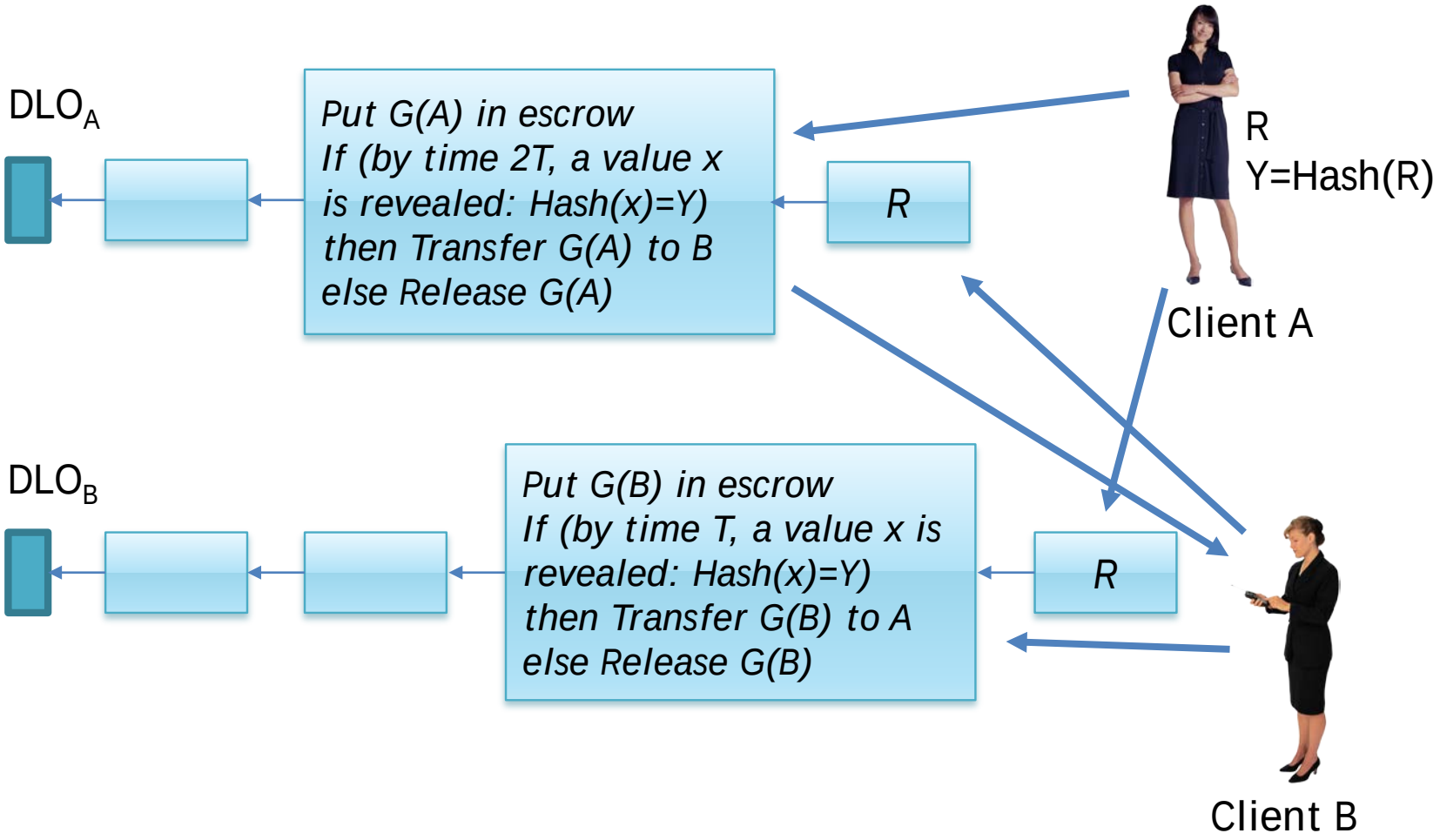
- Two selfish clients A and B want to exchange goods
- A has to transfer some good G(A) to B in blockchain DLO_A
- B has to transfer another good G(B) to A in blockchain DLO_B
- If any of them backs off, no transfer occurs



institute IMDEA networks

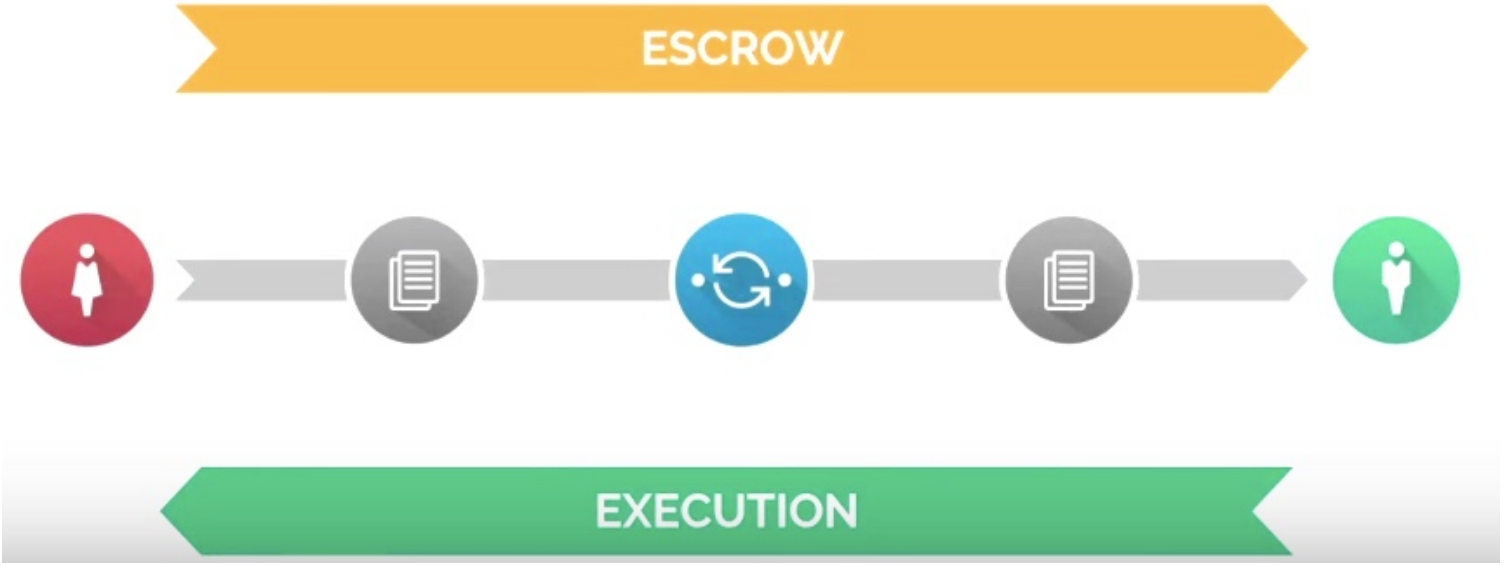
Atomic Swap with Hashlocks

- From [Narayanan et al 2016] [TierNolan 2013]:



More than Two Blockchains

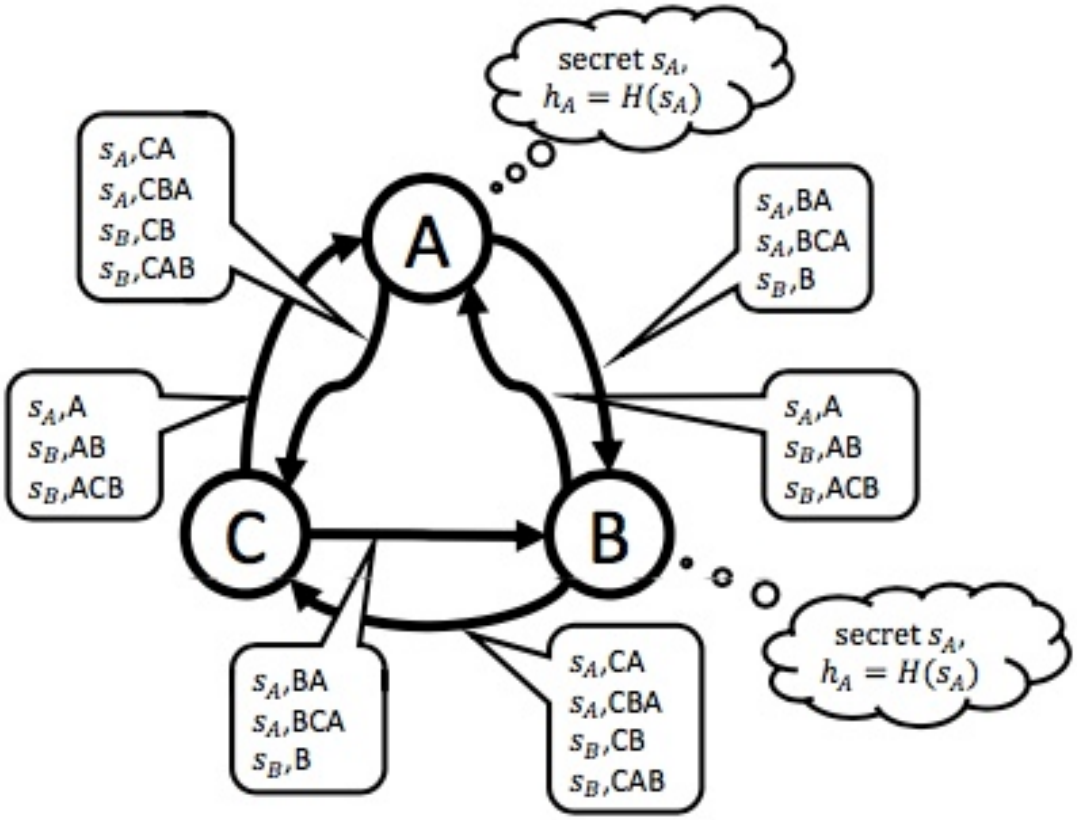
- Interledger [interledger.org] is a system that uses connectors, escrows, and hashlocks to propagate currency transfers across multiple blockchains



Source: Interledger Video https://www.youtube.com/watch?v=UdCxrqP6w3I

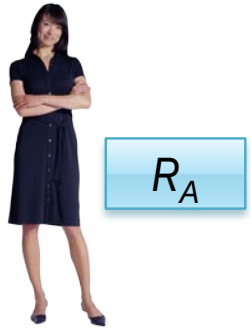
More than Two Blockchains

- [Herlihy 2018] generalizes this algorithm (and hashlocks) to the case in which the transfers form a strongly connected directed graph



Definition: Atomic Appends Problem

- Clients A and B
- Client A has record R_A to be appended to DLO_A
- Client B has record R_B to be appended to DLO_B
- Algorithm P solves the Atomic Appends Problem if
  - Safety: Either both records are appended or none
  - Liveness: If A and B do not fail, both records are appended



Client A



Client B

institute IMDEA networks

Competitive Utility: Selling Cars

- We assume clients A and B rational but risk-averse
- In the Competitive Utility Model,

$$U_X(\text{only Y appends}) > U_X(\text{both append}) > > U_X(\text{none appends}) > U_X(\text{only X appends})$$



institute IMDEA networks

Competitive Utility: Selling Cars

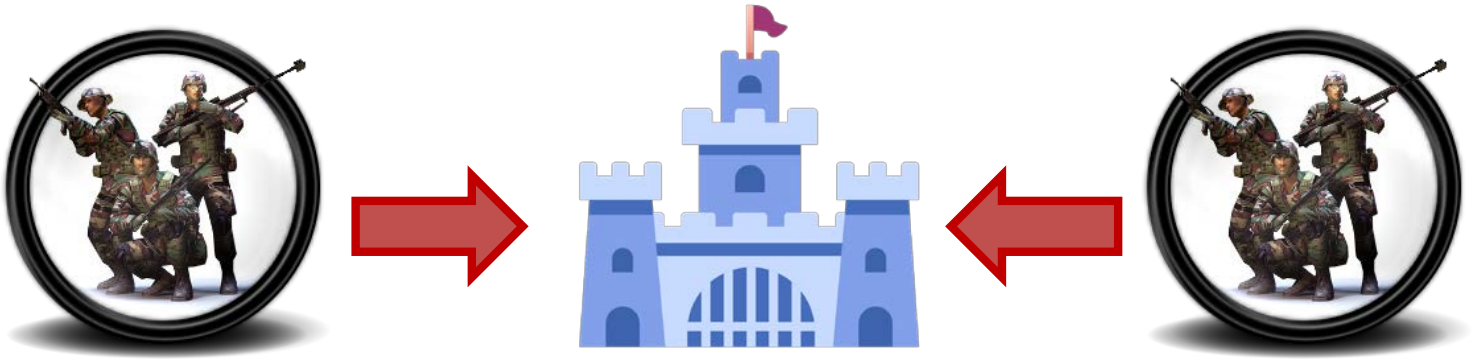
	Append	No
Append	( , )	(- ,  )
No	( , -)	( , )

- The Nash equilibrium is not appending
- Atomic Swap is an instance of Competitive Atomic Appends

institute IMDEA networks

Collaborative Utility: Coordinating Armies

- Consider the Coordinated Attack problem



- In the Collaborative Utility Model,

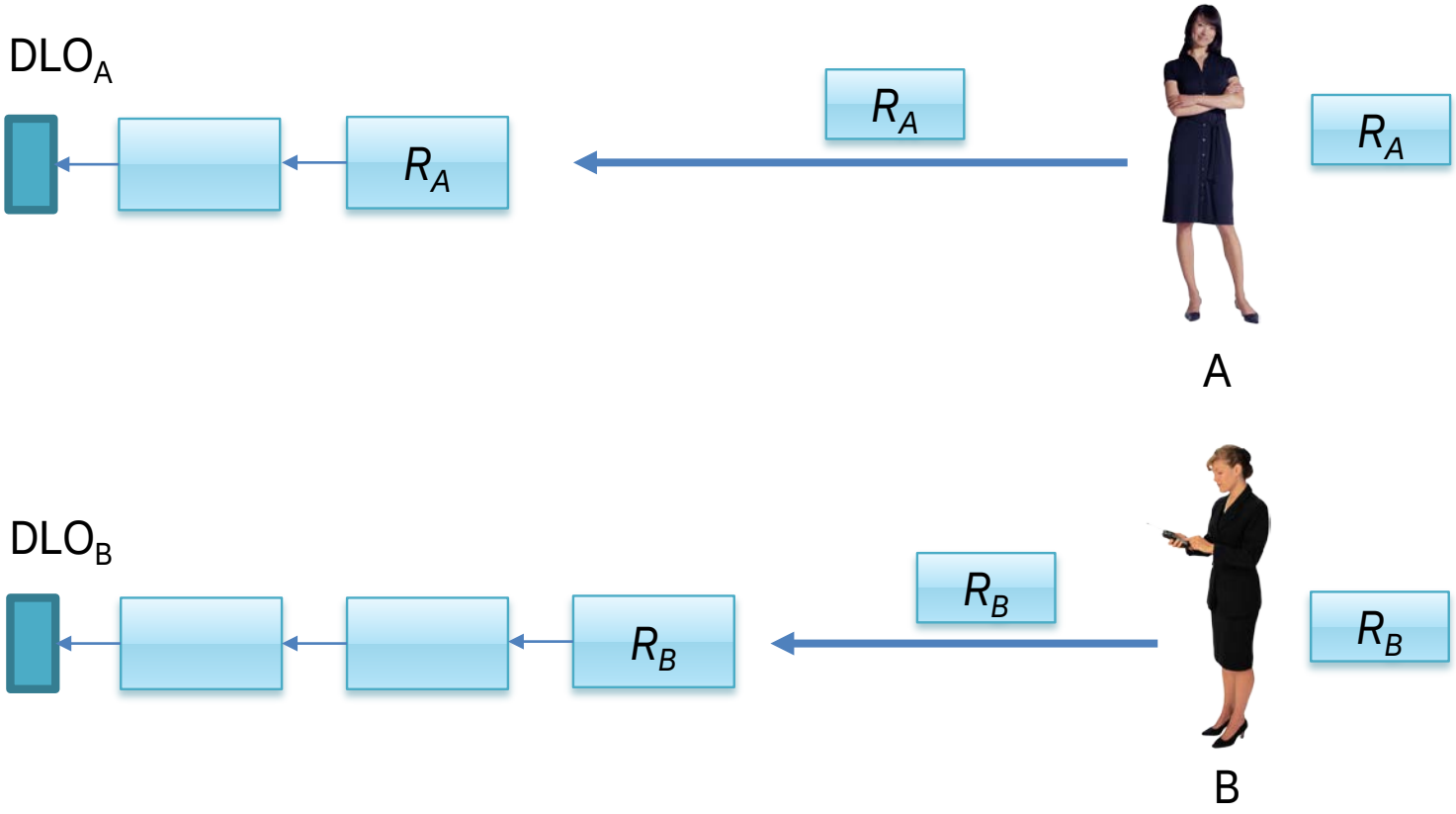
U_X(both append) > U_X(none appends) >

U_X(only Y appends) > U_X(only X appends)

institute IMDEA networks

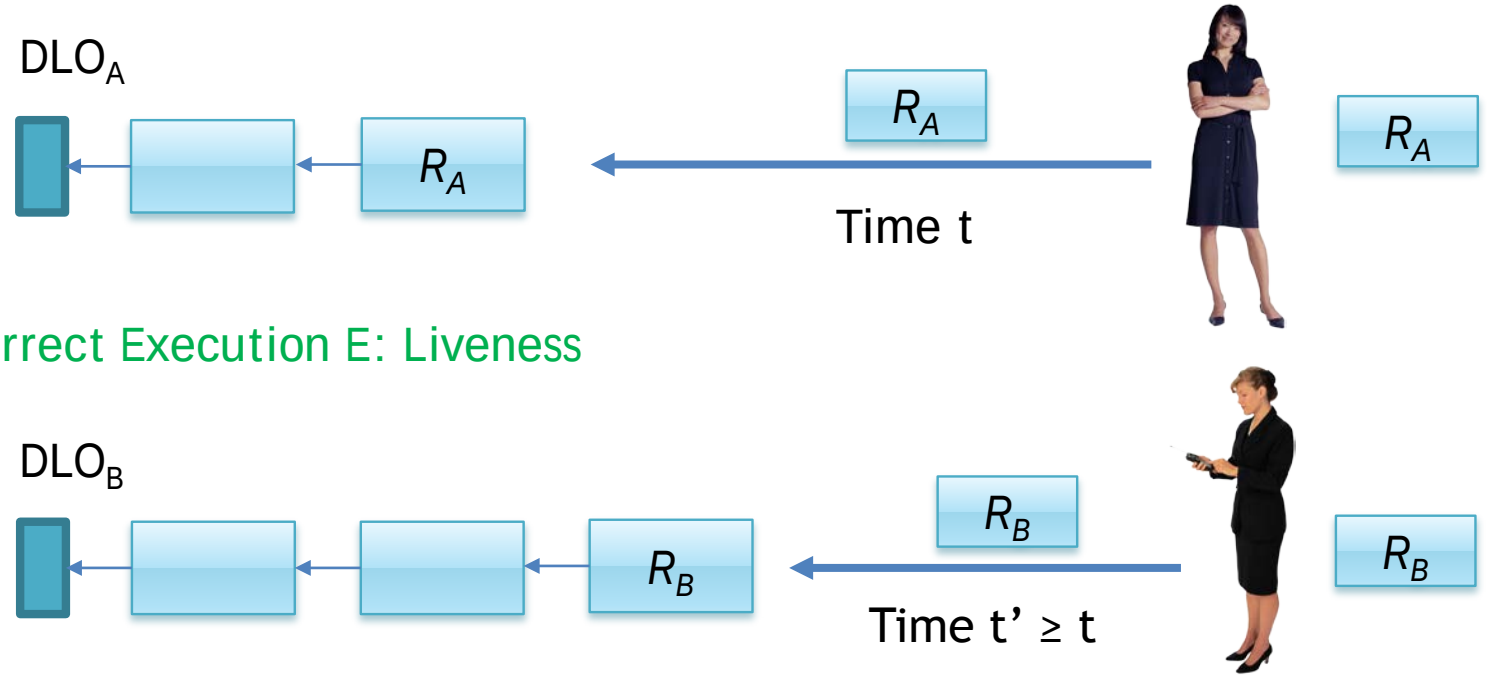
Collaborative Atomic Appends without Crashes

- In a failure-free system, both clients simply append



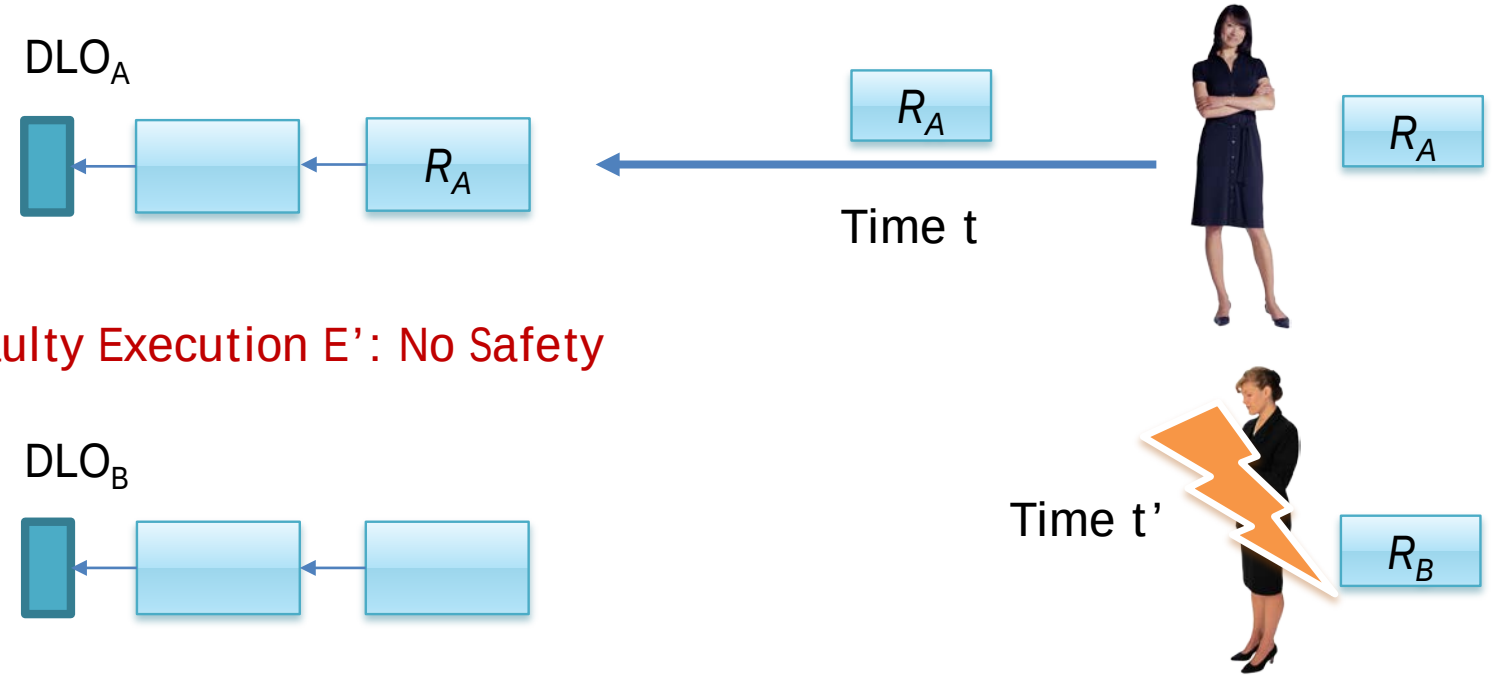
Collaborative Atomic Appends with Crashes

- Assume that clients can only append in their respective DLO (no delegation)
- Theorem: If clients can crash, Collaborative Atomic Appends cannot be solved, even in a synchronous system



Collaborative Atomic Appends with Crashes

- Assume that clients can only append in their respective DLO (no delegation)
- Theorem: If clients can crash, Collaborative Atomic Appends cannot be solved, even in a synchronous system



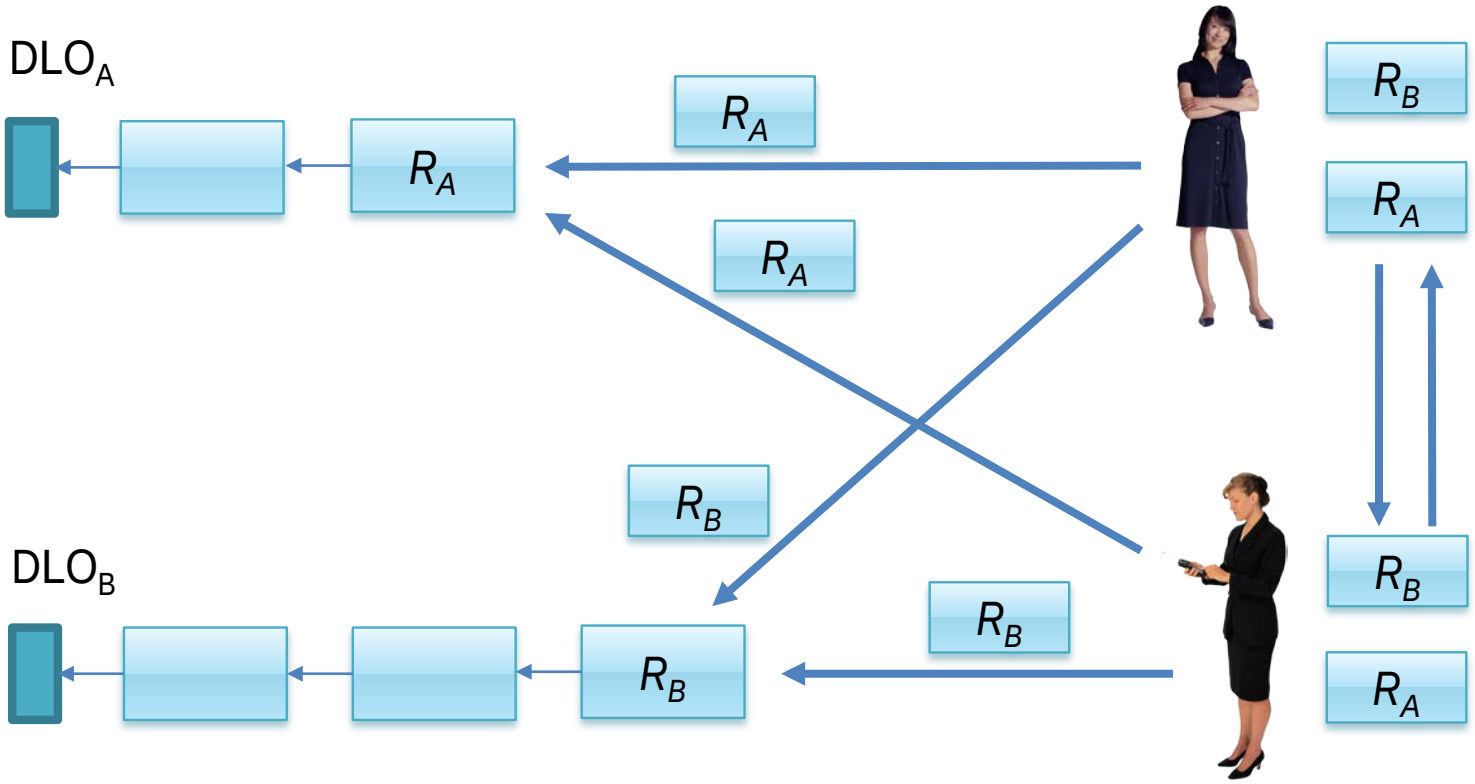
Delegation

- Let us now assume that delegation is possible (i.e., both clients can append in both DLOs)
- Algorithm A for an asynchronous system with delegation and one crash: Each client X does
  - Send own record R_X to the other client
  - Wait until record R_Y is received
  - Append record R_X
  - Append record R_Y

institute IMDEA networks

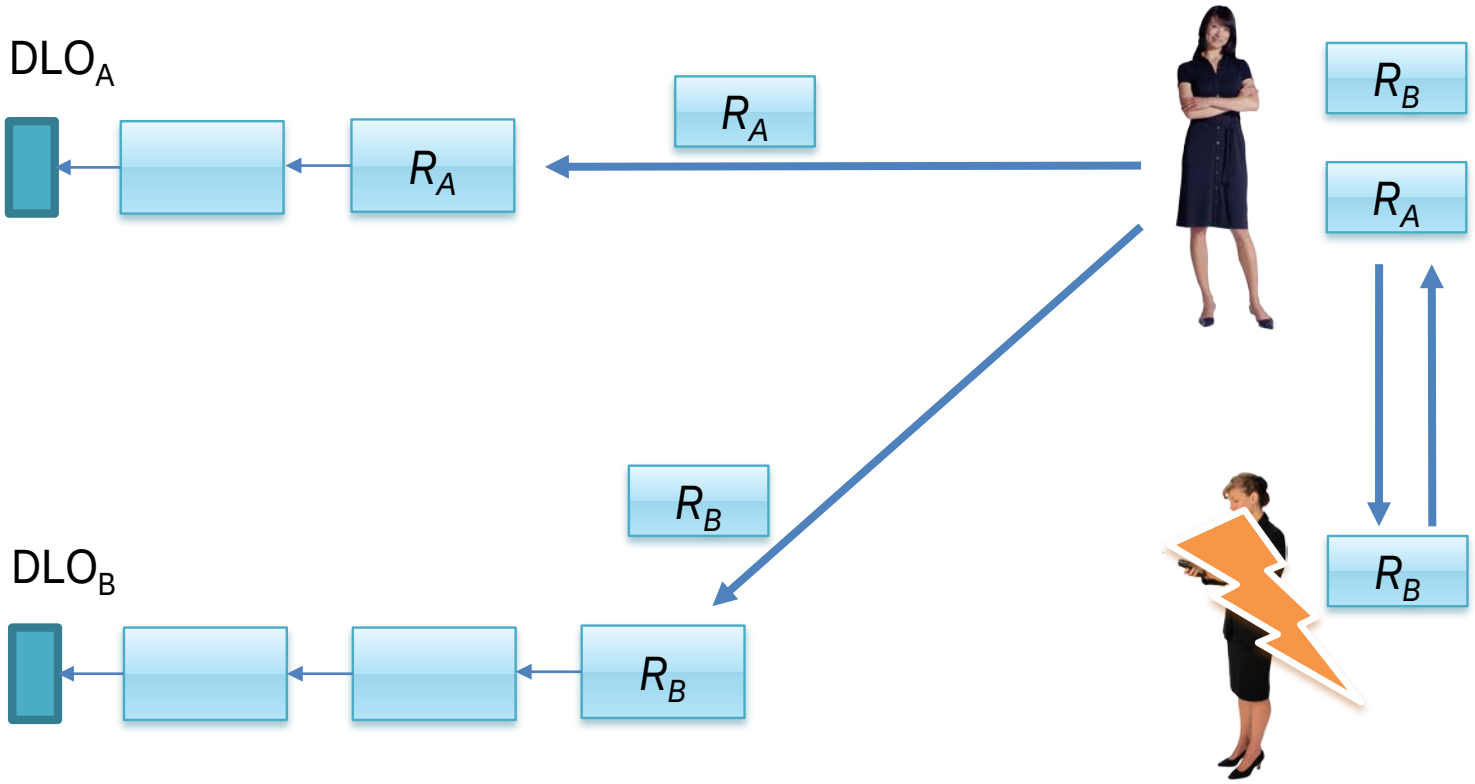
Algorithm A without Crashes

- Theorem: Algorithm A solves Collaborative Atomic Appends with delegation if at most one client crashes



Algorithm A with Late Crash

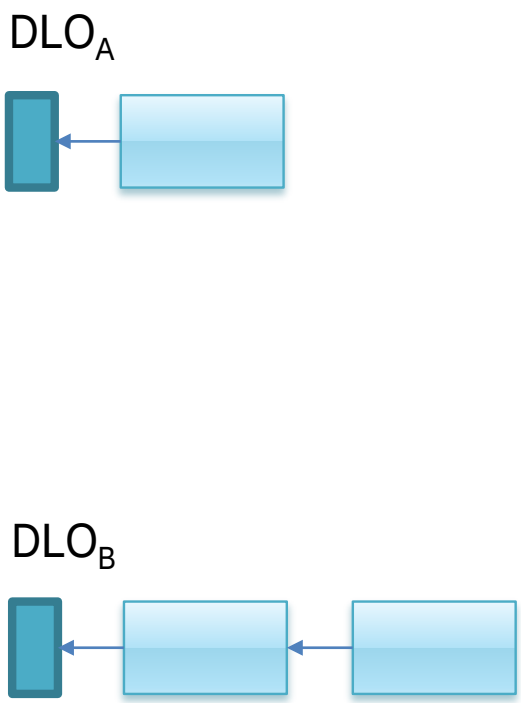
- Theorem: Algorithm A solves Collaborative Atomic Appends with delegation if at most one client crashes

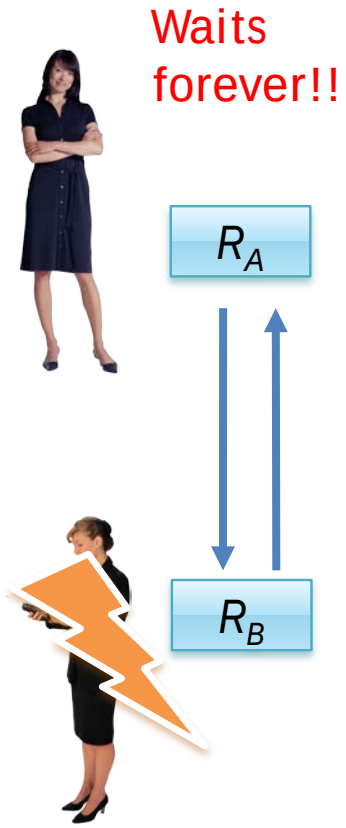


institute imdea networks

Algorithm A with Early Crash

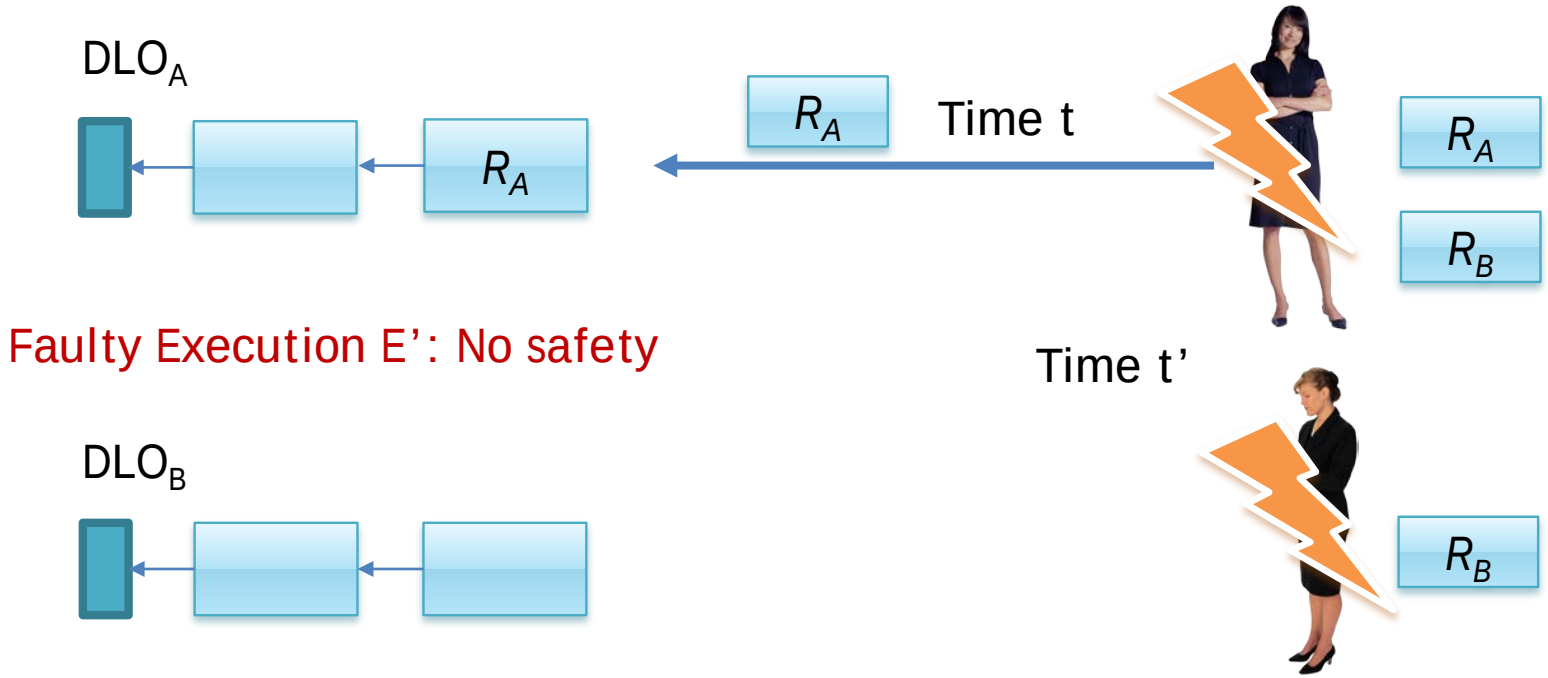
- Theorem: Algorithm A solves Collaborative Atomic Appends with delegation if at most one client crashes





Both Clients Can Crash

- Theorem: If both clients can crash, the problem cannot be solved, even with delegation and synchrony

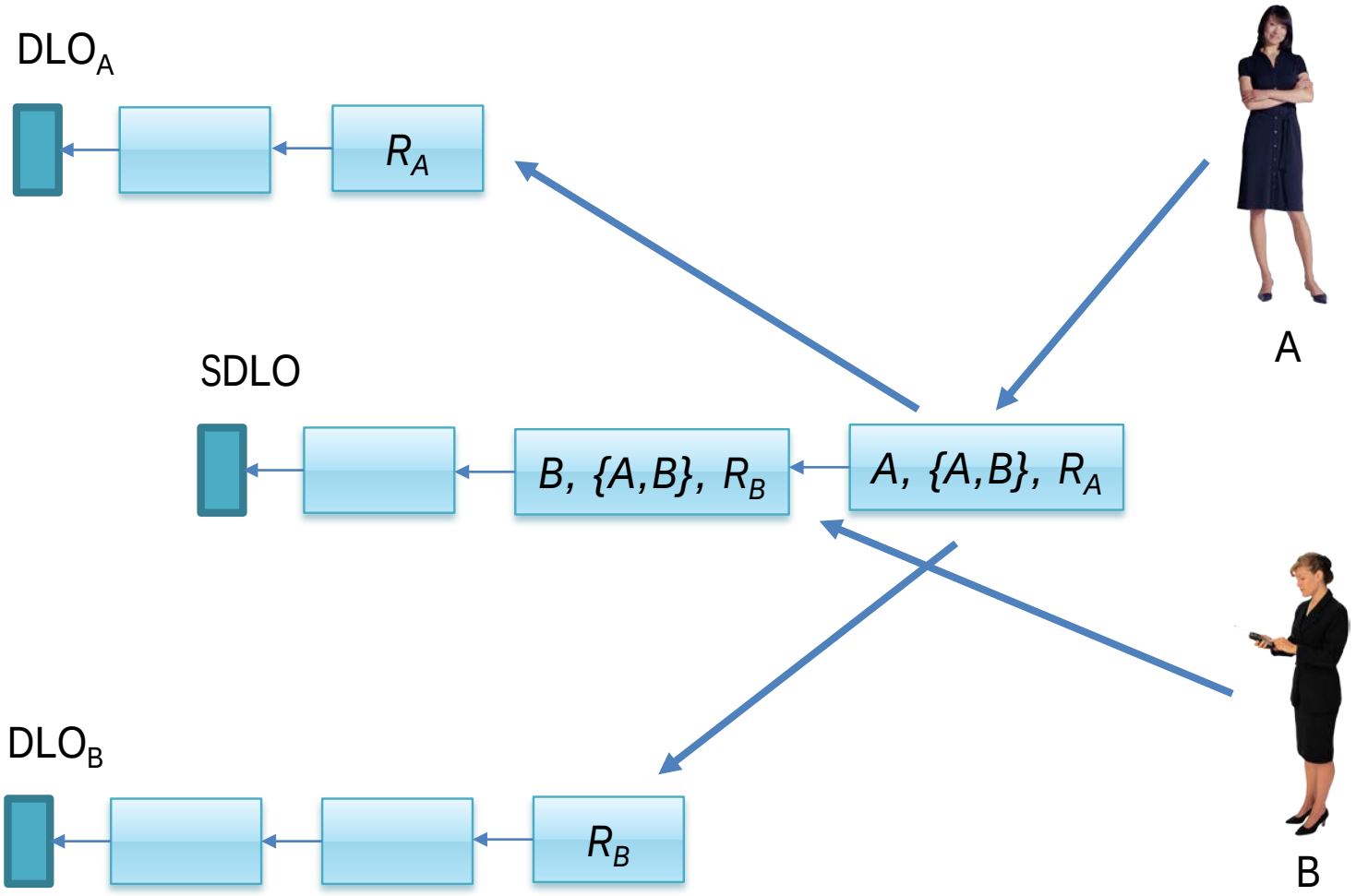


Delegation to a Smart DLO

- Delegating in each other is not enough if both clients can crash
- Can we do something else?
- We propose using a "Smart DLO" (SDLO) to collect the append requests and ultimately execute them:
  - Each client appends to the SDLO its description of the atomic append: e.g. [A, {A,B}, R_A]
  - When all required descriptions are appended in the SDLO, the SDLO appends in the corresponding DLOs

institute IMDEA networks

Delegation to a Smart DLO



institute IMDEA networks

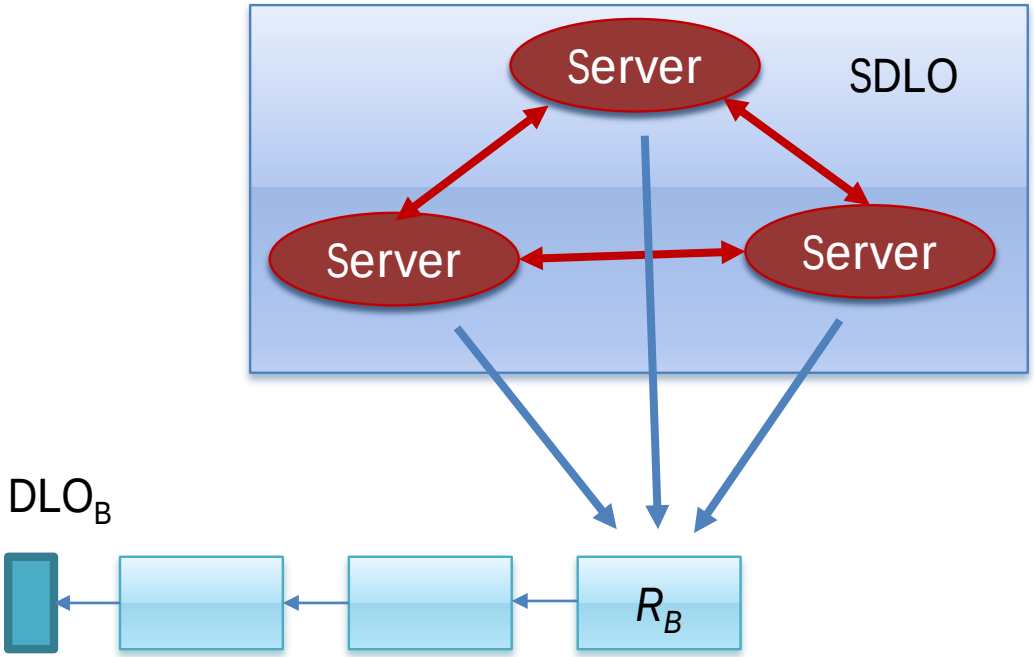
Solution with a Smart DLO

- Solves the problem even if both clients can crash and the system is asynchronous
- Works for any number of clients: *k*-Atomic Appends
- Can be used for Competitive Atomic Appends
- Assumes that the SDLO has functionality (a smart contract) that implements the algorithm
- The SDLO needs capacity of issuing *Append()* operations

institute IMDEA networks

Implementation of the Smart DLO

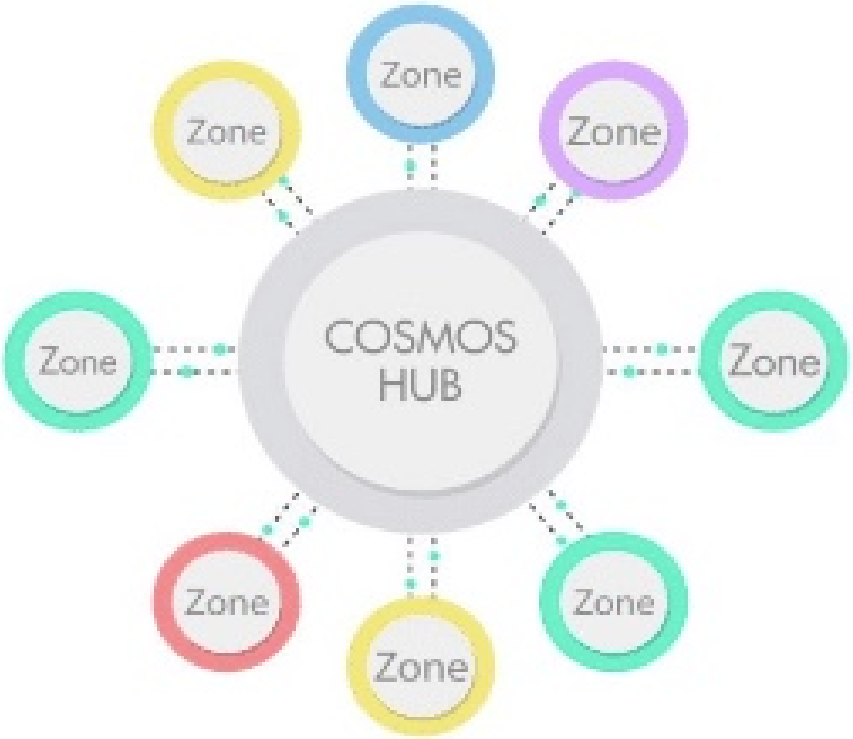
- The SDLO is not a real entity: implemented with multiple servers
- Must be able of appending in other DLOs!!



institute imdea networks

Other Interacting Blockchain Systems

- Cosmos: With a central blockchain (Hub)



Simple blockchain development

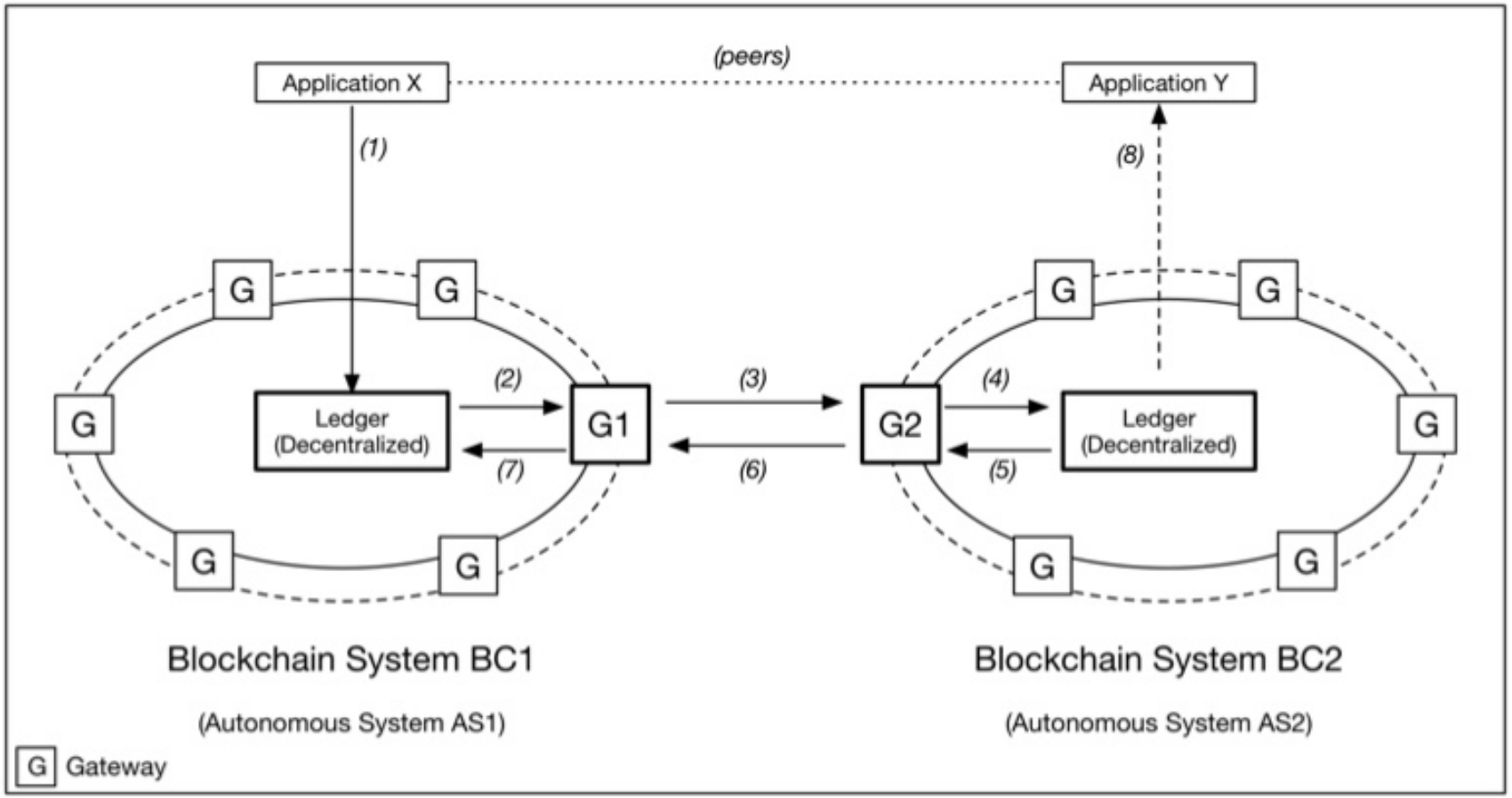
Smart contracts in any language

Byzantine fault tolerance

Source: Cosmos Video

Other Interacting Blockchain Systems

- Tradecoin [Hardjono et al 2018]: Gateways connect blockchains



Conclusions

- Isolated DLOs are not extremely useful
- Interconnection and interoperability between DLOs is needed
- We propose the Atomic Append problem, that already shows the challenges of interconnecting DLOs
- Collaborative Atomic Appends:

	No crash	1 crash	2 crashes
No delegation	YES	NO	NO
Delegation	YES	YES	NO
Smart DLO	YES	YES	YES

Works also for *k* clients and Competitive Atomic Appends

institute IMDEA networks

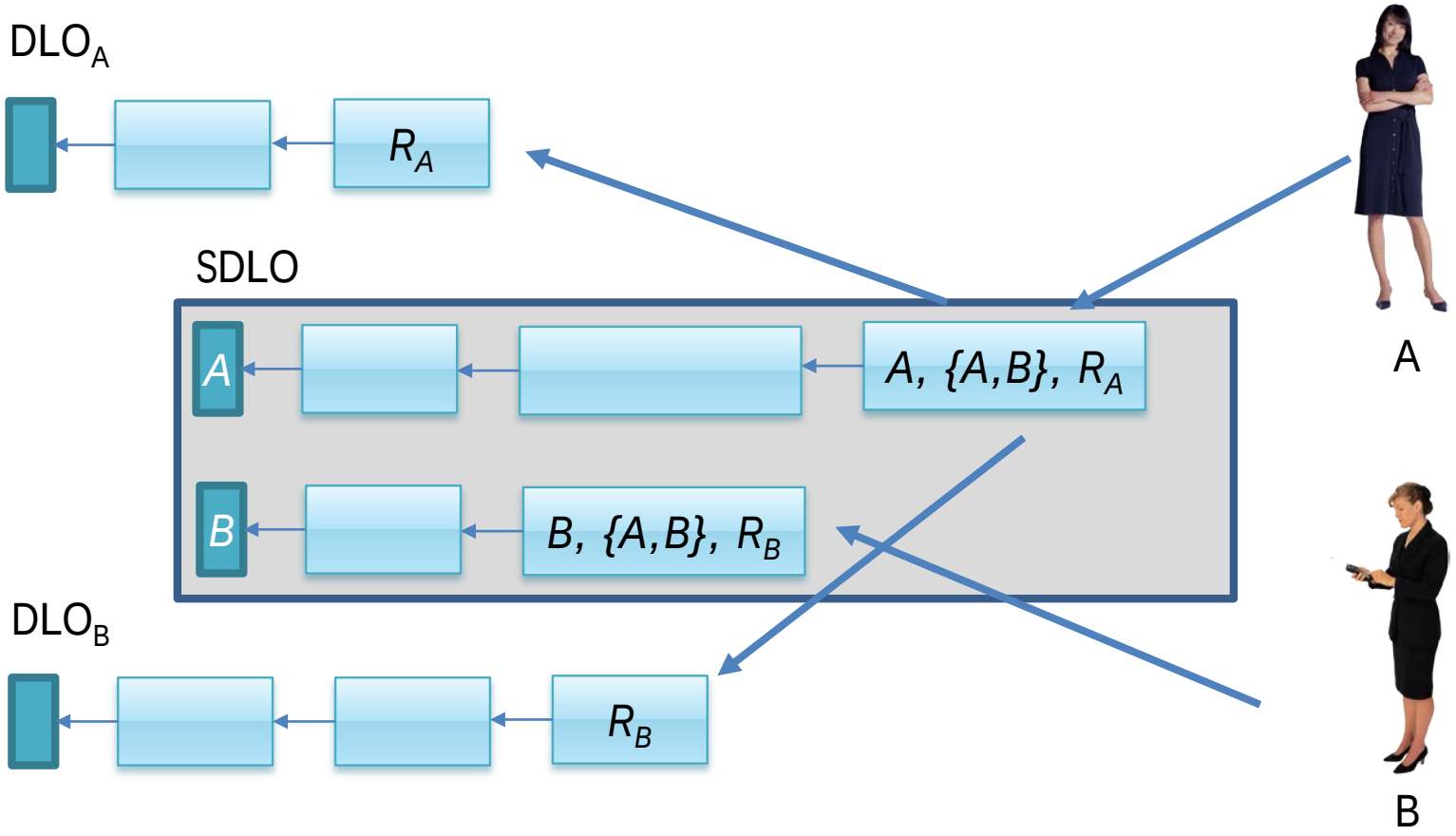
Future Work

- Explore all variants of Atomic Appends under different utility models
- Define Atomic Append as a native operation in a multi-ledger system:
  - CreateLedger(L)
  - Append(L,r)
  - Get(L)
  - AtomicAppend((L1,r1), (L2,r2), …)
- Implement DLOs that can access other DLOs
  - Read the records
  - Append records

institute IMDEA networks

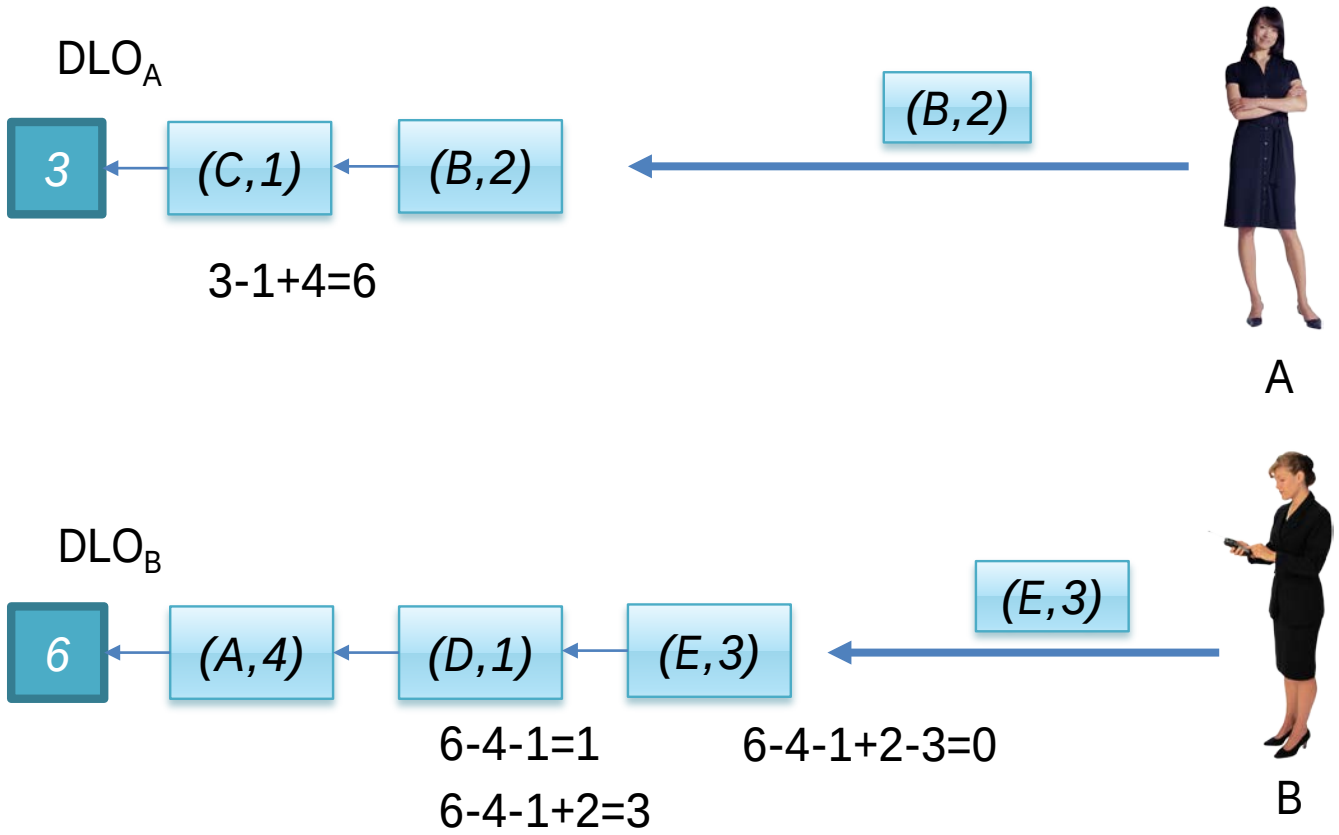
Future Work

- Design algorithms to implement Smart DLOs. Do we really need consensus? (Atomic Append descriptions do not need to be totally ordered)



Cryptocurrencies without Consensus

- Cryptocurrencies can be implemented without consensus [Gerraoui et al. PODC 2019]



Eskerrik asko!!

Thank you!!

Delegation

- Let us now assume that delegation is possible (i.e., both clients can append in both DLOs)
- Algorithm D for a synchronous system and one crash:
  - 1st round: A sends R_A to B; B sends R_B to A
  - 2nd and 3rd rounds: If client X received record R_Y in 1st round,
    - Then, append own record R_X, and then R_Y.
    - Otherwise, do nothing.

institute IMDEA networks

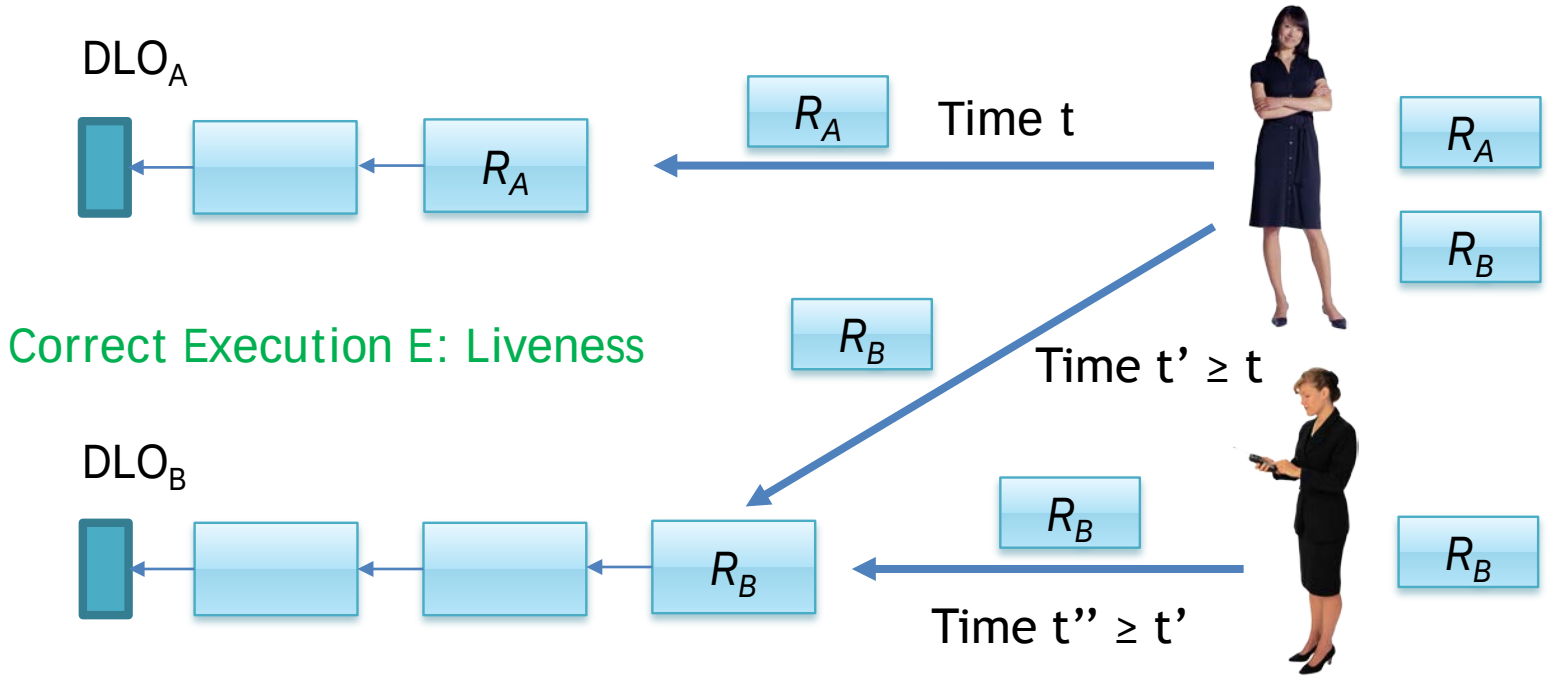
Asynchrony

- Algorithm A for an asynchronous system and one crash: Each client X does
  - Send own record R_X to the other client
  - Wait until record R_Y is received
  - Append record R_X
  - Append record R_Y
- Theorem: Algorithm A solves Collaborative Atomic Appends with delegation if at most one client crashes
- Proof: Case analysis
- Observe that the client may wait forever!!

institute IMDEA networks

Both Clients Can Crash

- Theorem: If both clients can crash, the problem cannot be solved, even with delegation and synchrony



Correct Execution E: Liveness

Both Clients Can Crash

- Theorem: If both clients can crash, the problem cannot be solved, even with delegation and synchrony

